

UNIT 2

FEATURE EXTRACTION & LANGUAGE MODELING

Application-Based Questions

Q1. A travel-booking chatbot recognises five intents forming the vocabulary {book, cancel, flight, hotel, refund}. Apply One-Hot Encoding to construct the vector for the intent "refund".

Solution 1

Vocabulary order: book, cancel, flight, hotel, refund (5 positions, $V = 5$).

Each word gets a vector of length 5 with a single 1 marking its own position and 0s elsewhere.

"refund" is the 5th word in the vocabulary, so its One-Hot vector is:

refund $\rightarrow [0, 0, 0, 0, 1]$

Q2. A recommendation engine must process 10,000 customer reviews of varying length and later needs to know how often each word is used. Determine which technique — One-Hot Encoding or Bag-of-Words — should be applied here, and justify your choice using the criteria of frequency capture and vector size.

Solution 2

Step 1 — Identify what the task needs: the model must later report how often each word is used, i.e. word frequency, not just presence/absence.

Step 2 — Compare the two techniques against this requirement: One-Hot Encoding only records whether a word is present (0 or 1) and loses all frequency information; Bag-of-Words counts actual occurrences of each word in each document.

Step 3 — Compare on vector size: both techniques use a fixed vocabulary-length vector once built, so both can represent variable-length documents as fixed-size vectors — this is not the deciding factor here.

Decision: Apply Bag-of-Words, because it is the only one of the two that preserves word frequency, which the recommendation engine explicitly needs.

Q3. A voice assistant is autocompleting the phrase "please turn on the ____". Apply the concept of a bigram language model to identify which single previous word the model would condition on, then propose two plausible next words with your reasoning.

Solution 3

Step 1 — Identify the model type: a bigram model conditions the next word on exactly one previous word, i.e. $P(\text{next word} \mid \text{previous word})$.

Step 2 — Identify the conditioning word: in "please turn on the ____", the single previous word immediately before the blank is "the", so the model computes $P(w \mid \text{the})$.

Step 3 — Propose candidates: two plausible completions are "lights" and "fan", since both commonly follow "the" in home-automation commands seen during training.

Step 4 — Prediction rule: the model compares $P(\text{lights} \mid \text{the})$ and $P(\text{fan} \mid \text{the})$ (and other candidates) and outputs whichever has the highest trained probability.

Q4. Using `CountVectorizer` with its factory defaults (`stop_words=None`, `lowercase=True`, tokens of 2+ alphanumeric characters), apply this configuration to determine the vocabulary that would be learned from the two sentences: "AI is transforming Healthcare!" and "Healthcare needs AI tools."

Solution 4

Step 1 — Apply `lowercase=True`: "AI" → "ai", "Healthcare" → "healthcare" in both sentences, so the two capitalisations of the same word are merged into one token.

Step 2 — Apply the token pattern (keep only tokens of 2+ alphanumeric characters): punctuation such as "!" is discarded; single-letter tokens would be dropped, but none occur here.

Step 3 — Collect the unique tokens across both sentences: ai, is, transforming, healthcare, needs, tools.

Step 4 — `CountVectorizer` sorts the vocabulary alphabetically, so the final learned vocabulary is: {ai, healthcare, is, needs, tools, transforming}.

Q5. Given `ngram_range=(1,2)` in `CountVectorizer`, apply this configuration to list every unigram and bigram feature that would be extracted from the sentence "smart phones are everywhere".

Solution 5

Step 1 — Tokenize the sentence: smart, phones, are, everywhere (4 tokens).

Step 2 — With `ngram_range=(1,2)`, extract all unigrams ($n=1$): smart, phones, are, everywhere.

Step 3 — Extract all bigrams ($n=2$) by sliding a 2-word window: "smart phones", "phones are", "are everywhere".

Step 4 — Combine both sets: the full feature set is {smart, phones, are, everywhere, smart phones, phones are, are everywhere} — 4 unigram + 3 bigram = 7 features.

Q6. A news aggregator wants to automatically tag articles with their most "distinctive" keyword. From the candidate words {the, is, economy, blockchain} appearing in a large multi-topic corpus, apply

TF-IDF scoring logic to determine which word would receive the highest TF-IDF weight in an article about cryptocurrency, and justify your answer.

Solution 6

Step 1 — Recall the TF-IDF logic: a word scores high when it is frequent within one document (high TF) but rare across the whole corpus (high IDF, i.e. low document frequency).

Step 2 — Evaluate each candidate against this logic: "the" and "is" are common function words that occur in nearly every document in a large multi-topic corpus, so their document frequency is very high and $IDF \approx 0$, driving their TF-IDF score toward 0 regardless of TF.

Step 3 — Compare "economy" and "blockchain": "economy" is a broadly used business term likely to appear across many articles (moderate-to-high DF, moderate IDF); "blockchain" is a specialised term that occurs in relatively few articles corpus-wide (low DF, high IDF) but would occur frequently within this specific cryptocurrency article (high TF).

Decision: "blockchain" would receive the highest TF-IDF weight in this article, since it combines high TF (frequent here) with high IDF (rare elsewhere).

Q7. Apply the Naive Bayes classification workflow (Prior $\rightarrow$ Likelihood $\rightarrow$ Evidence $\rightarrow$ Posterior) to outline, step by step, how you would build a sentiment classifier for movie reviews, without performing the numeric calculation.

Solution 7

Step 1 — Prior: compute $P(\text{Positive})$ and $P(\text{Negative})$ as the proportion of Positive and Negative reviews in the labelled training set.

Step 2 — Likelihood: for every word in the vocabulary, compute $P(\text{word} | \text{Positive})$ and $P(\text{word} | \text{Negative})$ from word counts within each class, applying Laplace smoothing so no probability is ever exactly zero.

Step 3 — Evidence/scoring: for a new review, treat every word as an independent feature (the naive assumption) and multiply $P(\text{class})$ by $P(\text{word} | \text{class})$ for each word in the review to obtain an unnormalised score per class.

Step 4 — Posterior/decision: compare $\text{Score}(\text{Positive})$ and $\text{Score}(\text{Negative})$ — optionally normalising by dividing by their sum (the evidence $P(X)$) — and assign the review to whichever class has the higher score.

Q8. Using the two sentences "Dog bites man" and "Man bites dog," construct their Bag-of-Words vectors and use the result to illustrate why applying plain BoW to a legal case-summarisation system could create serious errors.

Solution 8

Step 1 — Build the shared vocabulary from both sentences: {bites, dog, man} ($V = 3$, order-independent by definition of BoW).

Step 2 — Construct the BoW vector for "Dog bites man": bites=1, dog=1, man=1 $\rightarrow [1, 1, 1]$.

Step 3 — Construct the BoW vector for "Man bites dog": bites=1, dog=1, man=1 $\rightarrow [1, 1, 1]$.

Step 4 — Compare: the two vectors are identical, even though the sentences have opposite real-world meaning (who is doing the biting). In a legal case-summarisation system, this could make "Dog bit the man" and "Man bit the dog" appear as the same event to the model, potentially assigning liability to the wrong party — a serious, unacceptable error.

Q9. A search-engine autocomplete team needs a model that uses exactly two words of prior context to suggest the next word. Determine which model — unigram, bigram, or trigram — should be applied here, and justify your answer using the correct probability notation.

Solution 9

Step 1 — Restate the requirement: the model must use exactly two previous words as context before predicting the next word.

Step 2 — Match against model definitions: a unigram uses $P(w)$ (no context), a bigram uses $P(w_i | w_{i-1})$ (one previous word), and a trigram uses $P(w_i | w_{i-2}, w_{i-1})$ (two previous words).

Decision: the trigram model should be applied, since only $P(w_i | w_{i-2}, w_{i-1})$ conditions on exactly two prior words, matching the stated requirement exactly.

Q10. A machine-translation startup needs a model that can correctly relate the pronoun "it" to a noun that appeared 15 words earlier in the sentence. Apply your knowledge of generative language-model architectures to determine and justify the most suitable model, referencing the mechanism that allows it to succeed at this task.

Solution 10

Step 1 — Restate the requirement: the model must relate a pronoun to a noun 15 words earlier, i.e. it needs very long-range context.

Step 2 — Evaluate N-Gram: only looks at a small fixed window (e.g. 1–2 previous words) — cannot see back 15 words. Rejected.

Step 3 — Evaluate RNN/LSTM: carry a hidden memory state sequentially, but this memory tends to weaken over long sequences, especially in plain RNNs, making reliable 15-word-back retrieval unlikely.

Step 4 — Evaluate Transformer: uses self-attention to directly connect any two words in a sequence, regardless of distance, computing the relationship between "it" and its 15-words-earlier noun in a single step.

Decision: apply a Transformer-based model — its attention mechanism is specifically what makes long-distance word relationships tractable.

Q11. Construct the complete sets of Unigrams, Bigrams, and Trigrams for the phrase: "Artificial intelligence powers innovation."

Solution 11

Step 1 — Tokenize the phrase: Artificial, intelligence, powers, innovation (N = 4 words).

Step 2 — Unigrams (single words): {Artificial, intelligence, powers, innovation}.

Step 3 — Bigrams (2-word sliding window): {"Artificial intelligence", "intelligence powers", "powers innovation"}.

Step 4 — Trigrams (3-word sliding window): {"Artificial intelligence powers", "intelligence powers innovation"}.

Q12. A spam filter is trained on a vocabulary of 500 words. During deployment, an incoming email contains the word "cryptowallet," which never appeared in training. Apply your understanding of the Out-of-Vocabulary (OOV) problem to determine how a One-Hot/BoW/TF-IDF-based model would handle this word, and explain the likely impact on classification accuracy.

Solution 12

Step 1 — Recognise the situation: "cryptowallet" is an Out-of-Vocabulary (OOV) word — it never appeared in the 500-word training vocabulary used to fit the One-Hot/BoW/TF-IDF vectorizer.

Step 2 — Apply the OOV consequence: since the feature vector only has columns for the 500 training words, "cryptowallet" has no corresponding column and is simply dropped/ignored when the email is transformed into a vector.

Step 3 — Assess the impact: if "cryptowallet" was actually a strong indicator of spam, its signal is completely lost, which could cause the classifier to under-score the spam probability of this email and misclassify it as NotSpam.

Q13. You are designing an NLP pipeline to classify restaurant reviews as positive or negative. Apply the appropriate sequence of feature-extraction and classification techniques covered in this unit — from raw text to final label — listing each step in the order it should be executed.

Solution 13

Step 1 — Text cleaning/tokenization: lowercase the text, remove punctuation/noise, and split each review into tokens.

Step 2 — Build the vocabulary: collect all unique tokens (optionally including n-grams) across the training reviews.

Step 3 — Feature extraction: convert each review into a numeric vector using Bag-of-Words or TF-IDF (via CountVectorizer/TfidfVectorizer's fit_transform).

Step 4 — Train the classifier: fit a Naive Bayes (or other) classifier on the labelled (positive/negative) training vectors, learning priors and likelihoods.

Step 5 — Transform new data: apply the already-fitted vectorizer's transform() to any new review, using the same vocabulary learned in Step 2.

Step 6 — Predict: pass the new review's vector through the trained classifier to output the final positive/negative label.

Q14. A chatbot company observes that their bigram model assigns zero probability to several valid but rare customer queries, causing the chatbot to fail. Apply Add-One (Laplace) Smoothing conceptually to justify how it resolves this problem, referencing the smoothing formula in your answer.

Solution 14

Step 1 — Recall the problem: the bigram model assigns $P(w_i|w_{i-1}) = C(w_{i-1}, w_i) / C(w_{i-1}) = 0$ whenever a bigram was never seen in training, which wipes out the probability of any sentence containing it.

Step 2 — Apply Add-One (Laplace) Smoothing: modify the formula to $P(w_i|w_{i-1}) = (C(w_{i-1}, w_i) + 1) / (C(w_{i-1}) + V)$, adding 1 to every bigram count and V (vocabulary size) to every denominator.

Step 3 — Explain the effect: every possible bigram — including ones never observed in training — now receives a small but non-zero probability, so a rare-but-valid customer query is no longer assigned zero probability and the chatbot can still respond to it.

Q15. An internal document-search tool must rank 50,000 support articles by how distinctively each article uses a search term, rather than by raw word count. Determine which technique from this unit — Bag-of-Words or TF-IDF — should be applied, and justify your choice.

Solution 15

Step 1 — Identify the requirement: rank 50,000 articles by how *distinctively* a term is used, not merely by raw frequency.

Step 2 — Evaluate Bag-of-Words: only reflects frequency of a word within a document; a common word like "system" could dominate rankings purely because it is used often, even though it appears in almost every article and carries little distinguishing value.

Step 3 — Evaluate TF-IDF: multiplies term frequency (TF) by inverse document frequency (IDF), so words that are frequent in one article but rare across the other 49,999 receive a boosted score, while universally common words are suppressed.

Decision: apply TF-IDF, since it directly implements the "distinctiveness" ranking the search tool requires.

Numerical / Problem-Solving Questions

Q16. A support team logs three chat messages: M1: "The app crashes often" M2: "The app crashes rarely now" M3: "The website loads slowly" Apply the Bag-of-Words method to (a) build the vocabulary (list unique words alphabetically) and (b) construct the numerical vector for each message.

Solution 16

Vocabulary (alphabetical, 9 words): app, crashes, loads, now, often, rarely, slowly, the, website.

M1 "The app crashes often" $\rightarrow$ [app=1, crashes=1, loads=0, now=0, often=1, rarely=0, slowly=0, the=1, website=0] = [1,1,0,0,1,0,0,1,0]

M2 "The app crashes rarely now" $\rightarrow$ [1,1,0,1,0,1,0,1,0]

M3 "The website loads slowly" $\rightarrow$ [0,0,1,0,0,0,1,1,1]

Q17. For the document "Data science combines statistics data and programming" (treat words as case-insensitive), calculate the Term Frequency (TF) of every unique word in the document.

Solution 17

Lowercased tokens: data, science, combines, statistics, data, and, programming $\rightarrow$ Total words = 7.

Word counts: data=2, science=1, combines=1, statistics=1, and=1, programming=1.

TF(data) = $2/7 \approx 0.286$; TF(science) = TF(combines) = TF(statistics) = TF(and) = TF(programming) = $1/7 \approx 0.143$ each.

Q18. A corpus contains 4 documents. The word "cloud" appears in 3 of the 4 documents, and the word "quantum" appears in only 1 of the 4 documents. Using $IDF = \log_{10}(N / DF)$, calculate the IDF of "cloud" and of "quantum," and interpret which word is more distinctive.

Solution 18

N = 4 documents. DF(cloud) = 3 $\rightarrow$ IDF(cloud) = $\log_{10}(4/3) = \log_{10}(1.333) \approx 0.1249$.

DF(quantum) = 1 $\rightarrow$ IDF(quantum) = $\log_{10}(4/1) = \log_{10}(4) \approx 0.6021$.

Interpretation: "quantum" has a much higher IDF, so it is treated as far more distinctive/important than the common word "cloud".

Q19. Consider three documents: D1: "Data is powerful" D2: "Data is useful for science" D3: "Science is powerful and useful" Calculate the TF, IDF (base-10 log), and final TF-IDF score for every word in D2, and construct D2's TF-IDF vector.

Solution 19

Document word counts: D1 = 3 words, D2 = 5 words, D3 = 5 words. N = 3 documents.

Document Frequencies: data: D1,D2 $\rightarrow$ DF=2; is: D1,D2,D3 $\rightarrow$ DF=3; powerful: D1,D3 $\rightarrow$ DF=2; useful: D2,D3 $\rightarrow$ DF=2; for: D2 $\rightarrow$ DF=1; science: D2,D3 $\rightarrow$ DF=2.

IDF: data = $\log_{10}(3/2) = 0.176$; is = $\log_{10}(3/3) = 0$; useful = $\log_{10}(3/2) = 0.176$; for = $\log_{10}(3/1) = 0.477$; science = $\log_{10}(3/2) = 0.176$.

D2 = "Data is useful for science" — 5 words, each occurring once $\rightarrow$ TF = $1/5 = 0.20$ for every word.

TF-IDF (D2): data = $0.20 \times 0.176 = 0.0352$; is = $0.20 \times 0 = 0$; useful = $0.20 \times 0.176 = 0.0352$; for = $0.20 \times 0.477 = 0.0954$; science = $0.20 \times 0.176 = 0.0352$.

D2 TF-IDF vector (order data, is, useful, for, science) = [0.0352, 0, 0.0352, 0.0954, 0.0352]. "for" scores highest since it is rarest across the corpus (DF=1).

Q20. A training corpus contains three sentences (with boundary markers): S1: <s> I enjoy reading books </s> S2: <s> I enjoy watching movies </s> S3: <s> I enjoy reading news </s> Using the Bigram Maximum Likelihood Estimation formula $P(w_i|w_{i-1}) = C(w_{i-1}, w_i) / C(w_{i-1})$, calculate $P(\text{"I enjoy reading books"})$, showing every bigram count and conditional probability used.

Solution 20

Bigrams needed: <s> I, I enjoy, enjoy reading, reading books.

Count(<s>) = 3 sentences; Count(<s>, I) = 3 $\rightarrow P(I|<s>) = 3/3 = 1.0$

Count(I) = 3; Count(I, enjoy) = 3 $\rightarrow P(\text{enjoy}|I) = 3/3 = 1.0$

Count(enjoy) = 3; Count(enjoy, reading) = 2 (S1, S3) $\rightarrow P(\text{reading}|\text{enjoy}) = 2/3 \approx 0.667$

Count(reading) = 2; Count(reading, books) = 1 (S1) $\rightarrow P(\text{books}|\text{reading}) = 1/2 = 0.5$

$P(\text{"I enjoy reading books"}) = 1.0 \times 1.0 \times 0.667 \times 0.5 = 0.333 (\approx 33.3\%)$

Q21. Using the same training corpus as the previous question, with vocabulary $V = \{I, \text{enjoy, reading, watching, books, movies, news}\}$ ($V = 7$), calculate the Laplace-smoothed probability $P(\text{movies} | \text{reading})$, given that this bigram never occurred in the training data.

Solution 21

Count(reading, movies) = 0; Count(reading) = 2; $V = 7$.

$P(\text{movies}|\text{reading}) = (C(\text{reading, movies}) + 1) / (C(\text{reading}) + V) = (0+1) / (2+7) = 1/9 \approx 0.111$

Without smoothing this probability would have been exactly 0; smoothing gives it a small non-zero value instead.

Q22. Given the conditional probabilities $P(\text{The}) = 1.0$, $P(\text{weather} | \text{The}) = 0.4$, $P(\text{is} | \text{The, weather}) = 0.9$, and $P(\text{nice} | \text{The, weather, is}) = 0.6$, apply the Chain Rule of Probability to calculate $P(\text{The, weather, is, nice})$.

Solution 22

$P(\text{The, weather, is, nice}) = P(\text{The}) \times P(\text{weather}|\text{The}) \times P(\text{is}|\text{The, weather}) \times P(\text{nice}|\text{The, weather, is})$

$$= 1.0 \times 0.4 \times 0.9 \times 0.6 = 0.216 (\approx 21.6\%)$$

Q23. An email filter has Prior probabilities $P(\text{Spam}) = 0.25$ and $P(\text{NotSpam}) = 0.75$. For an incoming email $X = \text{"claim your reward,"}$ the Likelihoods are $P(X | \text{Spam}) = 0.7$ and $P(X | \text{NotSpam}) = 0.05$. Calculate the unnormalised scores for both classes, determine the final classification, and calculate the normalised posterior probability $P(\text{Spam} | X)$.

Solution 23

$$\text{Score}(\text{Spam}) = P(\text{Spam}) \times P(X|\text{Spam}) = 0.25 \times 0.7 = 0.175$$

$$\text{Score}(\text{NotSpam}) = P(\text{NotSpam}) \times P(X|\text{NotSpam}) = 0.75 \times 0.05 = 0.0375$$

Since $\text{Score}(\text{Spam}) > \text{Score}(\text{NotSpam})$, the email is classified as SPAM.

$$\text{Evidence } P(X) = 0.175 + 0.0375 = 0.2125$$

$$\text{Posterior } P(\text{Spam}|X) = 0.175 / 0.2125 \approx 0.8235 (\approx 82.4\%)$$

Q24. For the 7-word sentence "Machine learning models learn from data automatically," apply the formula $(N - n + 1)$ to calculate the number of bigrams, trigrams, and 4-grams that can be generated.

Solution 24

$$N = 7 \text{ words. Bigrams} = N - 2 + 1 = 6. \text{ Trigrams} = N - 3 + 1 = 5. \text{ 4-grams} = N - 4 + 1 = 4.$$

Q25. A binary sentiment training set has 8 words in the Positive class and 8 words in the Negative class (vocabulary size $V = 8$). The word "excellent" occurs 3 times in Positive reviews and 0 times in Negative reviews. Calculate the Laplace-smoothed $P(\text{excellent} | \text{Positive})$ and $P(\text{excellent} | \text{Negative})$, and state why smoothing was necessary here.

Solution 25

$$\text{Total words per class} = 8; V = 8; \text{smoothed denominator} = 8 + 8 = 16.$$

$$P(\text{excellent}|\text{Positive}) = (3+1)/16 = 4/16 = 0.25$$

$$P(\text{excellent}|\text{Negative}) = (0+1)/16 = 1/16 = 0.0625$$

Smoothing was necessary because "excellent" had a raw count of 0 in the Negative class, which without smoothing would have produced $P(\text{excellent}|\text{Negative}) = 0$ and could zero out the entire Negative-class score for any review

Conceptual Application Questions

Q26. In a 50-word product review, the word "excellent" appears twice. Apply both One-Hot Encoding and Bag-of-Words to determine the value each technique would assign to "excellent," and explain the practical consequence of this difference for a sentiment-scoring model.

Solution 26

Step 1 — Apply One-Hot Encoding: it only records presence/absence, so "excellent" gets the value 1, no matter how many times (2, in this case) it actually appears.

Step 2 — Apply Bag-of-Words: it counts true occurrences, so "excellent" gets the value 2, reflecting that it was used twice.

Step 3 — Consequence for sentiment scoring: a model built on One-Hot vectors treats a review that says "excellent" once the same as one that says it twice, losing the (often meaningful) signal that repeated strong positive words can indicate stronger sentiment. A BoW-based model preserves this signal and can weight the review more positively.

Q27. In a corpus of 200 documents, the word "the" appears in all 200 documents while the word "quantum" appears in only 2 documents. Apply the logic of the IDF formula to determine, without performing the full calculation, which word would be assigned greater importance by TF-IDF, and justify your reasoning.

Solution 27

Step 1 — Recall the IDF formula: $IDF = \log(N / DF)$; as DF (document frequency) decreases, N/DF increases, so IDF increases.

Step 2 — Apply it to "the": $DF = 200$ (appears in every one of the 200 documents), so $N/DF = 200/200 = 1$, and $IDF = \log(1) = 0$.

Step 3 — Apply it to "quantum": $DF = 2$ (appears in only 2 documents), so $N/DF = 200/2 = 100$, a large ratio, giving a much larger IDF than 0.

Conclusion: "quantum" is assigned far greater importance by TF-IDF than "the", because rarer words across the corpus receive higher IDF and therefore higher TF-IDF weight.

Q28. A dialogue system is designed to predict the next word using only the immediately preceding word. Apply the Markov Assumption to justify this design choice, and identify one risk this simplification introduces, using the sentence "I like data science" as your example.

Solution 28

Step 1 — Apply the Markov Assumption: it states that the next word depends only on a limited number of previous words, not the entire history — this justifies a bigram model using only $P(\text{word} \mid \text{previous word})$ instead of the full sentence context, keeping the model simple, fast, and requiring less training data.

Step 2 — Apply it to the example "I like data science": a bigram model predicting the word after "data" only looks at "data" itself and completely ignores "I" and "like".

Step 3 — Identify the risk: because the earlier words "I like" are discarded, the model may struggle to distinguish contexts like "data entry" versus "data science" that depend on cues further back in the sentence — the simplification trades some contextual accuracy for speed and simplicity.

Q29. Apply the Naive Bayes independence assumption to the phrase "not good service" by treating each word as an independent feature. Demonstrate why this assumption could cause the model to misclassify the sentiment of the phrase, and state the underlying limitation this exposes.

Solution 29

Step 1 — Apply the naive independence assumption: Naive Bayes treats every word in "not good service" as a separate, unrelated feature, computing something like $P(\text{not}|\text{class}) \times P(\text{good}|\text{class}) \times P(\text{service}|\text{class})$ rather than considering how "not" modifies "good".

Step 2 — Demonstrate the failure: because "good" alone is a strong indicator of the Positive class, its high $P(\text{good}|\text{Positive})$ can dominate the product even though the true phrase-level meaning ("not good") is negative — the negation carried by "not" is not linked to "good" in any way.

Step 3 — State the limitation: this exposes Naive Bayes's fundamental inability to model relationships or dependencies between words (such as negation, sarcasm, or word order), since its core assumption is that features are conditionally independent given the class.

Q30. A company is choosing between an N-Gram model and an LSTM for a customer-service chatbot that must remember information mentioned 10 conversational turns earlier. Apply the strengths-and-limitations comparison from this unit to determine and justify which architecture is better suited.

Solution 30

Step 1 — Restate the requirement: the chatbot must remember information mentioned 10 conversational turns earlier — a long-range dependency.

Step 2 — Evaluate the N-Gram model: it only conditions on a small, fixed number of previous words (typically 1–3), so information from 10 turns earlier is structurally impossible for it to access. Reject N-Gram.

Step 3 — Evaluate the LSTM: it is a neural network with a memory cell specifically engineered to retain relevant information over longer sequences and to selectively forget irrelevant details, unlike a plain N-Gram or vanilla RNN.

Decision: the LSTM should be chosen over the N-Gram model for this use case, although it should be noted that even LSTM has limits compared to attention-based Transformers for very long contexts.

Q31. Given the two short reviews "The food was good" and "The food was not good," apply Bag-of-Words vector construction to both, and use the resulting vectors to explain a fundamental weakness of frequency-based models in capturing negation.

Solution 31

Step 1 — Build the shared vocabulary: {food, good, not, the, was} ($V = 5$).

Step 2 — Construct the BoW vector for "The food was good": food=1, good=1, not=0, the=1, was=1 → [1, 1, 0, 1, 1].

Step 3 — Construct the BoW vector for "The food was not good": food=1, good=1, not=1, the=1, was=1 → [1, 1, 1, 1, 1].

Step 4 — Compare: the only difference between the two vectors is the "not" count (0 vs 1); every other word count is identical. This shows that a frequency-based model like BoW represents these two sentences as nearly identical, even though the second sentence has the opposite sentiment of the first — frequency counts alone cannot capture how negation changes meaning.

Q32. A CountVectorizer has already been fit() on a training corpus. Apply your understanding of the difference between fit() and transform() to determine what happens when transform() is called on a new document that contains a word absent from the fitted vocabulary.

Solution 32

Step 1 — Recall the distinction: fit() scans the training corpus and learns/stores the vocabulary (the fixed set of feature columns); transform() converts any given document into a numeric vector using that already-learned vocabulary.

Step 2 — Apply this to a new document: when transform() is called on text containing a word that was not present during fit(), there is no column reserved for that word in the output vector.

Step 3 — State the outcome: the unseen word is silently ignored (dropped) from the resulting vector — transform() does not raise an error, and it does not expand or update the previously learned vocabulary.

Q33. A language model is being trained on a very small corpus of only 200 sentences, where many valid word pairs are likely to be unseen. Apply your knowledge of smoothing techniques (Add-One/Laplace, Add-k, Good-Turing, Backoff, Interpolation) to recommend which technique(s) would be most suitable here, and justify your choice.

Solution 33

Step 1 — Assess the data situation: with only 200 training sentences, many valid bigrams/trigrams will simply never be observed, so the zero-probability problem will be frequent and severe.

Step 2 — Evaluate the candidate techniques: Good-Turing generally needs a reasonably large corpus to reliably re-estimate frequencies of rare events, making it less reliable here; Add-k Smoothing is a refinement of Add-One but still needs tuning; Backoff and Interpolation both let a higher-order model "fall back" on lower-order (more reliably estimated) n-gram probabilities when data is sparse.

Decision: Add-One (Laplace) Smoothing is the simplest and safest choice for very small corpora because it requires no tuning and guarantees non-zero probabilities everywhere; Backoff/Interpolation are strong complementary choices because they make better use of the limited data by relying on lower-order n-grams when higher-order counts are missing.

Q34. A financial-news classifier extracts the words "stock," "market," and "the" as features from a large corpus of financial articles. Apply your knowledge of TF-IDF weighting to determine which of the three words would most likely receive the lowest final TF-IDF score, and explain why.

Solution 34

Step 1 — Apply TF-IDF logic to "the": as a function word, it occurs in virtually every financial article, giving it a very high document frequency (DF) and therefore an IDF close to 0.

Step 2 — Apply TF-IDF logic to "stock" and "market": these are financial-domain terms that, while common in this corpus, still occur in a smaller proportion of documents than "the" and carry more topical meaning, so their IDF will be noticeably higher than "the"s.

Step 3 — Combine TF and IDF: since $TF\text{-}IDF = TF \times IDF$, and "the"s IDF is near 0 regardless of how often it appears in any single document, its final TF-IDF score will be the lowest of the three.

Conclusion: "the" would receive the lowest TF-IDF score, because TF-IDF specifically suppresses words that are common across nearly all documents in the corpus.

Case / Scenario-Based Questions

Q35. Case: An e-commerce platform collects three short reviews: R1: "Battery life is excellent" R2: "Camera quality is excellent and sharp" R3: "Battery drains fast" (a) Apply Bag-of-Words to construct the document-term matrix for R1, R2, and R3. (b) Using your matrix, determine which review shares the most vocabulary with R1, and justify your answer.

Solution 35

Vocabulary (alphabetical): and, battery, camera, drains, excellent, fast, is, life, quality, sharp → 10 words.

R1 "Battery life is excellent" → [and=0, battery=1, camera=0, drains=0, excellent=1, fast=0, is=1, life=1, quality=0, sharp=0]

R2 "Camera quality is excellent and sharp" → [and=1, battery=0, camera=1, drains=0, excellent=1, fast=0, is=1, life=0, quality=1, sharp=1]

R3 "Battery drains fast" → [and=0, battery=1, camera=0, drains=1, excellent=0, fast=1, is=0, life=0, quality=0, sharp=0]

(b) R2 shares the most vocabulary with R1: both contain "is" and "excellent" (2 shared words), while R3 shares only "battery" (1 shared word) with R1.

Q36. Case: A customer-support chatbot is trained on the sentences: "please cancel my order," "please track my order," and "please cancel my subscription." A customer types "please cancel my ____." (a) Apply the bigram MLE formula to calculate $P(\text{order} \mid \text{my})$ and $P(\text{subscription} \mid \text{my})$. (b) Determine which word the chatbot is more likely to predict next, showing your working.

Solution 36

Count(my) — from "cancel my order," "track my order," "cancel my subscription" — my is followed by: order (twice), subscription (once) $\rightarrow \text{Count}(\text{my}) = 3$.

$$(a) P(\text{order} | \text{my}) = \text{Count}(\text{my}, \text{order}) / \text{Count}(\text{my}) = 2/3 \approx 0.667$$

$$P(\text{subscription} | \text{my}) = \text{Count}(\text{my}, \text{subscription}) / \text{Count}(\text{my}) = 1/3 \approx 0.333$$

Q37. Case: A news portal wants to auto-generate a tag for each of three articles: A1: "Election results announced today" A2: "Sports team wins the championship today" A3: "Election campaign continues nationwide" Calculate the TF of "election" in A1 and the IDF of "today" (which appears in A1 and A2, out of 3 articles). Apply these results to determine whether "election" in A1 or "today" in A1 would receive the higher TF-IDF score, showing your calculation.

Solution 37

A1 "Election results announced today" has 4 words; "election" occurs once $\rightarrow \text{TF}(\text{election}, \text{A1}) = 1/4 = 0.25$.

"today" appears in A1 and A2 $\rightarrow \text{DF}(\text{today}) = 2, N = 3 \rightarrow \text{IDF}(\text{today}) = \log_{10}(3/2) \approx 0.176$.

"election" appears in A1 and A3 $\rightarrow \text{DF}(\text{election}) = 2, N = 3 \rightarrow \text{IDF}(\text{election}) = \log_{10}(3/2) \approx 0.176$ (same IDF as "today" here, since both occur in exactly 2 of 3 articles).

$\text{TF-IDF}(\text{election}, \text{A1}) = 0.25 \times 0.176 \approx 0.044$. $\text{TF}(\text{today}, \text{A1}) = 1/4 = 0.25 \rightarrow \text{TF-IDF}(\text{today}, \text{A1}) = 0.25 \times 0.176 \approx 0.044$.

Both words receive the same TF-IDF score in A1 (≈ 0.044) because they have identical TF and IDF in this case — the calculation shows students that TF-IDF depends on both factors together, not just on which word 'feels' more topic-specific.

Q38. Case: An email client's Smart-Compose feature is autocompleting "I hope you are doing ____." From its training logs, $\text{Count}(\text{"doing"}) = 500$, $\text{Count}(\text{"doing"}, \text{"well"}) = 350$, and $\text{Count}(\text{"doing"}, \text{"great"}) = 100$. Apply the bigram MLE formula to calculate $P(\text{well} | \text{doing})$ and $P(\text{great} | \text{doing})$, and determine which word the feature should suggest.

Solution 38

$$P(\text{well} | \text{doing}) = \text{Count}(\text{doing}, \text{well}) / \text{Count}(\text{doing}) = 350/500 = 0.70$$

$$P(\text{great} | \text{doing}) = \text{Count}(\text{doing}, \text{great}) / \text{Count}(\text{doing}) = 100/500 = 0.20$$

Since $P(\text{well} | \text{doing}) = 0.70 > P(\text{great} | \text{doing}) = 0.20$, Smart Compose should suggest "well."

Q39. Case: An email service deploys spam detection using CountVectorizer features and a Naive Bayes classifier. Given $P(\text{Spam}) = 0.3$ and $P(\text{NotSpam}) = 0.7$, and an incoming email containing the word "lottery" with $P(\text{lottery} | \text{Spam}) = 0.6$ and $P(\text{lottery} | \text{NotSpam}) = 0.02$: (a) Apply Bayes' Theorem to

calculate the unnormalised score for each class.(b) Determine the final classification.(c) Calculate the normalised posterior probability $P(\text{Spam} | X)$.

Solution 39

$$(a) \text{Score}(\text{Spam}) = P(\text{Spam}) \times P(\text{lottery}|\text{Spam}) = 0.3 \times 0.6 = 0.18$$

$$\text{Score}(\text{NotSpam}) = P(\text{NotSpam}) \times P(\text{lottery}|\text{NotSpam}) = 0.7 \times 0.02 = 0.014$$

(b) Since $0.18 > 0.014$, the email is classified as SPAM.

(c) Evidence $P(X) = 0.18 + 0.014 = 0.194$. Posterior $P(\text{Spam}|X) = 0.18/0.194 \approx 0.928$ ($\approx 92.8\%$)

Q40. Case: A voice-assistant company must choose an architecture to generate multi-sentence responses that stay coherent across long conversations (up to 50 words of context). They are evaluating N-Gram, RNN, LSTM, and Transformer models from this unit. Apply the comparison criteria (context length, ability to relate distant words, computational cost) to determine and justify the most suitable architecture, and explain why the two weakest options would fail for this use case.

Solution 40

The Transformer should be chosen. Its self-attention mechanism directly relates any two words in the sequence regardless of distance, giving it the strongest ability to stay coherent across 50 words of context. N-Gram models fail because they only look at a fixed, very short window of previous words and cannot reference anything from far earlier in the conversation. Plain RNNs fail because their hidden state degrades over long sequences, losing information from many turns earlier (the reason LSTM and then attention-based Transformers were developed).

